

HAMP-LIC: Hessian-Aware Mixed-Precision Post-Training Quantization for Learned Image Compression

Yuefeng Zhang

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

Abstract—Learned image compression (LIC) models achieve strong rate–distortion performance but are hindered by high computational complexity and encoding–decoding mismatches across heterogeneous hardware platforms. Uniform fixed-precision quantization alleviates these issues but suffers severe quality degradation at low bit-widths, as it ignores the differing quantization sensitivities of individual layers. To enable efficient and accurate low-bit deployment of pre-trained LIC models, we propose HAMP-LIC, a Hessian-aware mixed-precision post-training quantization (PTQ) framework with a four-stage optimization strategy. First, block-wise sensitivity is estimated from the Hessian trace to capture second-order importance. Second, a task-aware refinement module adjusts these sensitivities by jointly considering quantization distortion and rate–distortion performance. Third, guided by the refined sensitivity profile, bit-widths are allocated under a global model-size constraint to balance efficiency and reconstruction quality. Finally, block-wise reconstruction on a small calibration set further suppresses quantization error. Experiments on representative LIC models, including *Minnen2018* and *Cheng2020*, demonstrate that HAMP-LIC achieves up to $4.85\times$ model compression with as low as 0.59% BD-rate loss, consistently outperforming existing fixed- and mixed-precision PTQ methods across multiple datasets while completely eliminating cross-platform encoding–decoding errors.

Index Terms—Learned image compression, post-training quantization, mixed-precision quantization, Hessian-based sensitivity analysis, model compression.

I. INTRODUCTION

Recent advances in learned image compression (LIC) have established it as a leading paradigm in image and video coding, achieving significant improvements in rate–distortion performance over traditional handcrafted codecs. By jointly optimizing nonlinear transforms and probabilistic entropy models, LIC systems are able to learn compact latent representations that substantially enhance compression efficiency while preserving reconstruction quality. Despite these advantages, the computational and memory requirements of modern LIC architectures remain prohibitive for deployment on resource-constrained devices, limiting their practical applicability in real-world scenarios.

In addition to high inference cost, LIC models are also sensitive to numerical precision [1]. In practice, different hardware platforms exhibit inconsistent behavior in floating-point arithmetic, particularly in entropy modeling and probability

Yuefeng Zhang is with the Beijing Institute of Computer Technology and Application, Beijing, China. She is currently pursuing a Ph.D. degree at the School of Elite Engineering, Northwestern Polytechnical University, Shaanxi, China (e-mail: yuefeng.zhang@mail.nwpu.edu.cn).

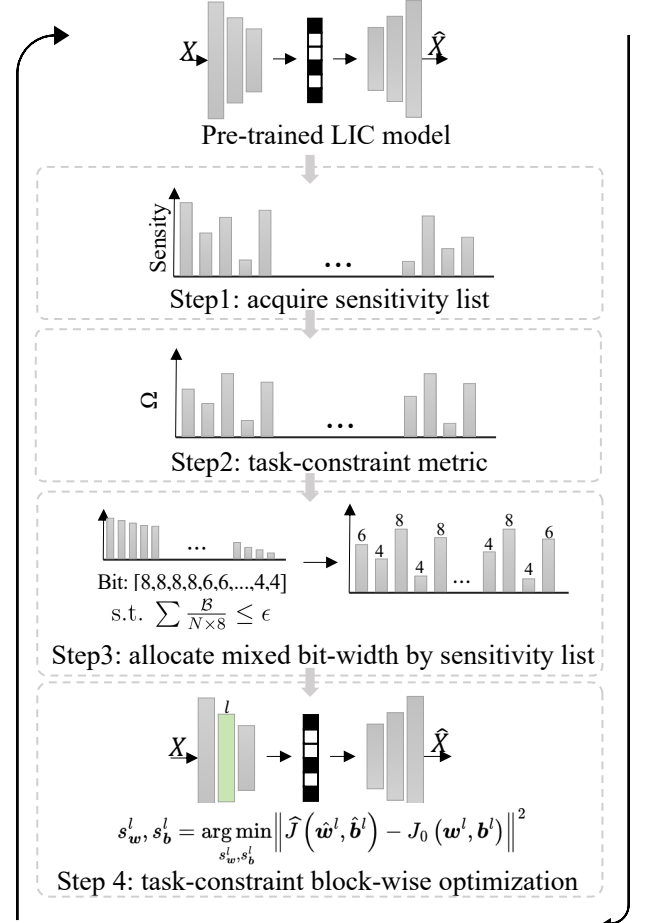


Fig. 1. The overview of the proposed quantization framework HAMP-LIC for the pre-trained LIC model, where 4 steps are conducted.

estimation. Such discrepancies can lead to encoding–decoding mismatches, resulting in degraded reconstruction quality or even instability across platforms. This problem is far from negligible in practice: as shown in our experiments (Section IV-B4), a full-precision *Cheng2020* model fails to decode up to 24 out of 24 Kodak images and 98 out of 100 Tecnick images when encoding and decoding are performed on different devices (CPU and GPU). These issues highlight the need for efficient and numerically robust compression strategies that maintain performance under low-precision computation.

To address these challenges, model quantization has emerged as a promising solution for reducing computational complexity and memory footprint. Existing quantization methods are generally categorized into Quantization-Aware Train-

ing (QAT) [2] and Post-Training Quantization (PTQ) [3]–[6]. QAT incorporates quantization effects during training to improve robustness but requires full access to training data and substantial retraining cost. By treating all modules with a fixed precision, these methods ignore the heterogeneous sensitivity of different modules, which often leads to suboptimal rate–distortion performance under aggressive quantization. Although the recent FMPQ [6] explores mixed-precision quantization for LIC, it remains limited to relatively high precision (e.g., 8-bit average) and does not fully investigate ultra-low-bit regimes. In contrast, PTQ directly converts a pre-trained FP32 model into a low-bit representation using a small calibration set, eliminating training time. However, most existing PTQ approaches for LIC adopt uniform bit-width assignment across all parameters: Hong *et al.* [3] restrict fixed-point inference to the decoder with range-adaptive quantization (RAQ), He *et al.* [4] first apply PTQ to pre-trained LIC networks without re-training, and RDO-PTQ [5] further introduces rate–distortion-aware block-wise reconstruction.

From a broader perspective, existing LIC quantization methods still face a fundamental trade-off between efficiency, accuracy, and generality. Uniform quantization strategies, while simple, fail to reflect the highly non-uniform redundancy across network components, leading to inefficient precision allocation. In contrast, mixed-precision approaches developed for general vision tasks are often tailored to classification networks and rely on heuristic sensitivity metrics or expensive search procedures, making them difficult to transfer to LIC settings with rate–distortion optimization and entropy-constrained latent representations. Moreover, LIC models possess unique structural characteristics, such as hyperprior-based entropy modeling and iterative analysis–synthesis transforms, which further complicate sensitivity estimation and bit allocation. These differences highlight the necessity of a dedicated mixed-precision quantization strategy specifically designed for LIC.

To further improve efficiency and robustness, recent studies have shown that second-order information, particularly the Hessian matrix [7], provides a principled measure of layer-wise sensitivity in neural networks. This insight enables more informed precision allocation by identifying modules that are more sensitive to quantization perturbations. Motivated by this observation, we propose a mixed-precision PTQ framework for LIC that enables efficient low-bit compression while ensuring numerical consistency across heterogeneous hardware platforms.

In our method, block-wise sensitivity is estimated using Hessian-based second-order information, and bit-width allocation is formulated as a constrained integer optimization problem under a target compression ratio. The optimization further incorporates rate–distortion-aware objectives to jointly minimize quantization error and task-specific distortion. The proposed framework, termed Hessian-Aware Mixed-Precision PTQ for LIC (HAMP-LIC), follows a four-stage pipeline (Fig. 1): (1) sensitivity estimation, (2) task-aware metric construction, (3) mixed-precision allocation via constrained optimization, and (4) block-wise reconstruction refinement.

This article substantially extends our DCC 2026 paper, MPP-LIC [8]. Beyond the four-stage mixed-precision PTQ

framework, it introduces task-aware sensitivity combining Hessian-trace information with rate–distortion degradation, formulates bit-width allocation as a global size-constrained optimization solved via Pareto-frontier search, and details block-wise reconstruction using sequential scaling optimization and adaptive rounding. The main contributions of this work are summarized as follows:

- 1) We propose HAMP-LIC, a mixed-precision PTQ framework that integrates Hessian-based second-order sensitivity estimation with rate–distortion-aware optimization for LIC models.
- 2) We design a task-aware sensitivity refinement that modulates the Hessian-trace metric with the rate–distortion loss degradation of each block, so that bit-width decisions directly reflect the compression objective rather than generic reconstruction error.
- 3) We formulate bit allocation as a constrained integer optimization problem and introduce an efficient Pareto-frontier search strategy that reduces complexity from exponential to tractable scale, combined with progressive block-wise refinement.
- 4) We demonstrate that HAMP-LIC achieves up to $4.85\times$ model compression with negligible BD-rate degradation, while ensuring cross-platform numerical consistency and eliminating floating-point-induced decoding mismatches.

The remainder of this paper is organized as follows. Section II reviews related work on neural network compression, quantization for LIC, and mixed-precision quantization. Section III details the four steps of the proposed HAMP-LIC framework. Section IV presents the experimental settings, comparisons with state-of-the-art PTQ methods, and ablation studies. Finally, Section V concludes this paper.

II. RELATED WORK

In this section, we first review neural network compression methods in Section II-A, followed by applications of model compression in LIC in Section II-B. Finally, we discuss mixed-precision quantization methods in Section II-C.

A. Neural Network Compression

Neural network compression aims to reduce computational cost and memory footprint while preserving model accuracy. Among existing techniques, quantization is particularly attractive due to its hardware efficiency, converting full-precision weights and activations into low-bit representations.

Quantization methods are typically categorized into Quantization-Aware Training (QAT) [9], [10] and Post-Training Quantization (PTQ) [11], [12]. QAT simulates quantization during training, allowing the model to adapt to quantization errors via gradient-based optimization [2]. While effective under extremely low bit-widths, it requires full training data and incurs high computational cost. PTQ, in contrast, directly quantizes pre-trained models using a small calibration set to estimate activation statistics [4]. Although efficient, PTQ often suffers from noticeable accuracy degradation in low-bit settings.

Recent reconstruction-based approaches aim to bridge this gap by optimizing quantization with unlabeled calibration data. AdaRound [13] formulates weight rounding as a learnable optimization problem. BRECQ [14] performs block-wise reconstruction to capture inter-layer dependencies. AdaQuant [15] jointly optimizes weight rounding and activation clipping, while QDrop [16] improves robustness by stochastically bypassing quantization during reconstruction. These methods operate at different granularities (e.g., tensor-, layer-, and block-level), balancing optimization flexibility and computational efficiency.

B. Application of Model Compression in LIC

Model compression, particularly quantization, has become increasingly important in LIC to enable efficient deployment. Early work focused on mitigating the inconsistency of floating-point computation across hardware. Ballé et al. [2] proposed integer-only LIC networks to ensure deterministic and hardware-friendly inference. Hong et al. [3] further explored partial quantization by restricting fixed-point inference to the decoder, reducing complexity while preserving rate-distortion performance.

More recent efforts have shifted toward fully quantized LIC models. He et al. [4] first applied Post-Training Quantization (PTQ) to pre-trained LIC networks, enabling efficient deployment without retraining, but suffering from performance degradation at low bit-widths. To address this, Sun et al. [17] reduced activation dynamic ranges via channel splitting and pruning, improving quantization robustness.

Recent advances incorporate reconstruction-based PTQ into LIC. Shi et al. [5] integrated BRECQ [14] for block-wise reconstruction and further introduced task-aware rate-distortion optimization, moving beyond MSE-based objectives. These efforts reflect a shift toward structured and task-aware quantization, narrowing the performance gap with full-precision LIC while maintaining efficiency.

C. Mixed Precision Quantization

Mixed-precision quantization exploits the observation that different layers in a neural network exhibit varying sensitivity to quantization [7], [18]. Instead of applying a uniform bit-width across all layers, it assigns higher precision to more sensitive layers and lower precision to less sensitive ones, thereby achieving a better trade-off between model size, computational efficiency, and accuracy. This adaptive allocation is particularly beneficial for deep architectures, where quantization errors can accumulate unevenly across layers.

Most existing approaches [19], [20] formulate mixed-precision quantization as a global optimization problem, aiming to determine the optimal bit-width configuration for each layer under constraints such as model size, latency, or energy consumption. These methods typically integrate bit-width selection into the training process, leveraging gradient-based optimization or reinforcement learning to jointly optimize network weights and precision assignments. While effective, such approaches often incur substantial computational overhead due to the large search space.

A key challenge arises from the combinatorial nature of the problem: the search space for layer-wise bit-width assignment grows exponentially with the number of layers, making exhaustive search intractable for modern deep networks. To alleviate this issue, sensitivity-based methods [7], [21], [22] have been proposed. These approaches first estimate the quantization sensitivity of each layer in a pre-trained model, typically based on metrics such as reconstruction error or loss perturbation. Bit-widths are then allocated by ranking layers according to their sensitivity, assigning higher precision to layers that are more critical to overall performance. This strategy significantly reduces the search complexity while maintaining competitive accuracy.

III. PROPOSED METHOD

In this section, we present the proposed HAMP-LIC framework. We first introduce the preliminaries (Section III-A), including the block-wise quantization formulation and the rate-distortion objective of LIC models, and then detail the four steps ((Sections III-B–III-E)) through which HAMP-LIC applies mixed-precision quantization to the pretrained LIC model. The complete procedure is summarized in Algorithm 1.

A. Preliminaries

Assume that the LIC model can be partitioned into L blocks denoted by $\{B_1, B_2, \dots, B_L\}$, each associated with learnable weight parameters $\{W_1, W_2, \dots, W_L\}$. Each block may consist of one or more layers, and the quantization of each block is performed independently to enable fine-grained reconstruction.

The b -bit uniform quantization operation applied to a weight parameter W is defined as:

$$W_b = \text{Clip}(\lfloor W/s \rfloor, n, p) \quad (1)$$

where s denotes the quantization scale parameter that controls the mapping from floating-point values to integers, and n and p represent the negative and positive integer clipping thresholds, respectively, which are determined by the target bit-width b as $n = -2^{b-1}$ and $p = 2^{b-1} - 1$ for signed quantization. The rounding operation $\lfloor \cdot \rfloor$ maps each scaled value to its nearest integer, and the clipping operation $\text{Clip}(\cdot, \cdot)$ ensures that all values remain within the representable range. Following previous studies [5], [17], we adopt per-channel scale quantization, where each output channel of a weight tensor is assigned an independent scale parameter s , allowing for finer adjustment of the quantization range and reducing the overall quantization error compared to per-tensor quantization.

For the LIC network, the overall training objective can be formulated as a rate-distortion loss function defined over the training set:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N J(x_i, \hat{x}_i; \theta) \quad (2)$$

where $x \in X$ denotes the input image set, $\hat{x} \in \hat{X}$ denotes the corresponding reconstructed image set produced by the LIC model, and $N = |X|$ is the total number of training samples. The per-sample R-D loss $J(x_i, \hat{x}_i; \theta)$ combines a

Algorithm 1 HAMP-LIC: Hessian-Aware Mixed-Precision PTQ for LIC

Require: FP32 model $\mathcal{M} = \{B_l\}_{l=1}^L$; calibration set $\mathcal{X}_{\text{cal}}$; bit-width set B ; budget ϵ .

Ensure: Mixed-precision quantized model $\mathcal{M}_{mp}$.

```

1: // 1. Hessian-based sensitivity
2: for  $l = 1$  to  $L$  do
3:   Estimate  $\text{Tr}(H_l)$  on  $\mathcal{X}_{\text{cal}}$  using Hutchinson's method;
4: end for
5: // 2. Build task-oriented sensitivity list
6: Evaluate the 8-bit reference loss  $\mathcal{L}_{\text{int8}}$  on  $\mathcal{X}_{\text{cal}}$ ;
7: for  $l = 1$  to  $L$  do
8:   for  $b \in B$  do
9:     Quantize  $B_l$  to  $b$  bits and evaluate  $\mathcal{L}_{\text{qp},l}(b)$ ;
10:     $\Omega_l(b) \leftarrow \text{Tr}(H_l) \frac{|\mathcal{L}_{\text{qp},l}(b) - \mathcal{L}_{\text{int8}}|}{|\mathcal{L}_{\text{int8}}|}$ ;
11:   end for
12: end for
13: // 3. Mixed-precision allocation
14: Rank blocks by task-aware sensitivity and enumerate
    monotone contiguous partitions as in Eq. (7);
15:  $\{b_l^*\}_{l=1}^L \leftarrow \underset{\{b_l\}}{\text{argmin}} \sum_{l=1}^L \Omega_l(b_l)$ 
16:   s.t.  $b_l \in B$ ,  $\frac{1}{8L} \sum_{l=1}^L b_l \leq \epsilon$ ;
17: // 4. Block-wise reconstruction
18: for  $l = 1$  to  $L$  do
19:   Set  $\hat{w}^l = Q(w^l; s_w^l)$  and  $\hat{b}^l = Q(b^l; s_b^l)$  using  $b_l^*$  bits;
20:   Optimize  $(s_w^{l*}, s_b^{l*})$  by minimizing  $(\hat{J} - J_0)^2$ ;
21:   Optimize  $V^{l*}$  and the scale factors by minimizing
      $\lambda_t \mathcal{L}_{\text{task}} + \mathcal{L}_{lq}$ ;
22: end for
23: return  $\mathcal{M}_{mp}$  with  $\{b_l^*\}_{l=1}^L$ ,  $s^*$ , and  $V^*$ .
```

distortion term measuring reconstruction fidelity (e.g., MSE or MS-SSIM) and a rate term penalizing the estimated bitrate of the latent representation, jointly encouraging the model to achieve a favorable rate-distortion trade-off.

B. Step 1: Acquire Sensitivity List

Our approach begins with a full-precision pre-trained LIC network $\mathcal{M}$ and constructs a per-block sensitivity list to guide the subsequent mixed-precision quantization process. The sensitivity list characterizes how sensitive each block is to quantization-induced perturbations, enabling the allocation of higher bit-widths to more sensitive blocks and lower bit-widths to more robust ones.

A straightforward choice for measuring block sensitivity is the first-order gradient information. However, first-order metrics are often insufficient for accurately capturing layer-wise sensitivity, as they do not account for the curvature of the loss landscape [7]. To address this limitation, we adopt second-order information as the primary sensitivity metric. Specifically, we utilize the Hessian trace of the loss function with respect to the weights of each block, which reflects the

local curvature and provides a more reliable indicator of how much the model output would change in response to weight perturbations introduced by quantization.

Formally, for block B_l with weight parameters W_l , the Hessian matrix is defined as:

$$H_l = \frac{\partial^2 \mathcal{L}}{\partial W_l^2} \quad (3)$$

where $l \in \{1, \dots, L\}$ and $\mathcal{L}$ is the loss function defined over the calibration set. The Hessian trace $\text{Tr}(H_l)$ serves as the sensitivity score for block B_l , with a larger trace value indicating greater sensitivity to quantization [7].

Direct computation of the full Hessian matrix is computationally prohibitive for large-scale neural networks. To ensure computational tractability, we instead estimate the Hessian trace efficiently via Hessian-vector products combined with randomized probing techniques, based on Hutchinson's method [7], [23].

Once the Hessian trace is computed for all L blocks, the sensitivity scores are collected and ranked to form the sensitivity list $\mathcal{S} = \{(B_l, \text{Tr}(H_l))\}_{l=1}^L$, which serves as the foundation for bit-width allocation in the subsequent step.

C. Step 2: Build Task-Oriented Sensitivity List

While the Hessian trace obtained in Step 1 provides a geometry-aware measure of each block's sensitivity to weight perturbations, it does not directly reflect the impact of quantization on the downstream compression task. To address this, we introduce a task-constrained sensitivity metric that couples the second-order curvature information with the task-specific rate-distortion loss, enabling a more accurate assessment of each block's contribution to the overall model performance under quantization.

Specifically, for each block B_l we define a task-aware sensitivity Ω_l as a composite measure that integrates both the potential sensitivity captured by the Hessian trace and the task-oriented loss degradation induced by quantization. Formally, Ω_l is expressed as:

$$\Omega_l(b_l) = \text{Tr}(H_l) \cdot \frac{|\mathcal{L}_{\text{qp}}(b_l) - \mathcal{L}_{\text{int8}}|}{|\mathcal{L}_{\text{int8}}|} \quad (4)$$

where L is the number of quantizable blocks, b_l is the weight bit-width assigned to the l -th block, $\text{Tr}(H_l)$ is the Hessian trace of block B_l computed in Step 1, and $|\cdot|$ denotes the absolute value. The term $\mathcal{L}_{\text{qp}}(b_l)$ denotes the task loss evaluated when the weights of block B_l are quantized to b_l bits, while $\mathcal{L}_{\text{int8}}$ denotes the task loss of the reference model whose weights are uniformly quantized to 8 bits. The ratio $|\mathcal{L}_{\text{qp}}(b_l) - \mathcal{L}_{\text{int8}}|/|\mathcal{L}_{\text{int8}}|$ measures the relative task loss degradation introduced by quantizing block B_l , which serves as a task-aware weighting factor to modulate the Hessian-based sensitivity. By combining these two terms, Ω_l captures both the geometric sensitivity of each block and its practical impact on the compression objective.

For the i -th sample of an LIC model with a hyperprior structure, the rate-distortion (R-D) loss $J_i = J(x_i, \hat{x}_i; \theta)$ is defined as:

$$\begin{aligned} J_i &= \lambda \cdot D_i + R_{\hat{Y}_i} + R_{\hat{Z}_i} \\ &= \lambda \cdot D(x_i, \hat{x}_i) \\ &\quad + \mathbb{E} \left[-\log_2 \left(p_{\hat{Y}}(\hat{Y}_i | \hat{Z}_i) \right) \right] + \mathbb{E} \left[-\log_2 \left(p_{\hat{Z}}(\hat{Z}_i) \right) \right], \end{aligned} \quad (5)$$

where λ is the Lagrange multiplier that controls the trade-off between rate and distortion, $R_{\hat{Y}_i}$ is the estimated bitrate for the quantized latent representation $\hat{Y}_i$, and $R_{\hat{Z}_i}$ is the estimated bitrate for the hyperprior latent representation $\hat{Z}_i$. $D(x_i, \hat{x}_i)$ denotes the distortion between the original image x_i and its reconstruction $\hat{x}_i$, which is typically measured by MSE or MS-SSIM depending on the target application.

Given the task-aware sensitivities $\{\Omega_l\}$ defined in Eq. (4), the quantization problem is formulated as a constrained optimization that seeks the optimal per-block bit-width assignment $\{b_l\}_{l=1}^L$ to minimize the accumulated sensitivity-weighted loss degradation—an efficient proxy for the R-D loss J —while satisfying the model compression ratio budget:

$$\begin{aligned} &\underset{\{b_l\}}{\operatorname{argmin}} \sum_{l=1}^L \Omega_l(b_l) \\ \text{s.t. } &\frac{1}{8L} \sum_{l=1}^L b_l \leq \epsilon \end{aligned} \quad (6)$$

where ϵ is a hyperparameter controlling the overall compression ratio relative to 8-bit quantization, which is empirically set to $\epsilon = 0.75$ in our experiments; a 10% tolerance on the average bit-width is allowed in practice when enumerating candidate allocations. This constraint ensures that the average bit-width across all blocks does not exceed a prescribed budget, effectively limiting the model size overhead introduced by mixed-precision quantization. The detailed optimization procedure for solving this constrained problem is described in Section III-D.

D. Step 3: Mixed-Precision Bit-Width Allocation via Pareto Frontier Search

Given the task-aware sensitivities $\{\Omega_l\}$ refined in Step III-C, this step solves the constrained optimization problem in Eq. (6) to determine the bit-width assignment for each block. Let B denote the set of admissible bit-widths of size k for each block, e.g., $B = \{2, 4, 8\}$, and the bit-width b_l assigned to each block in Eq. (6) must satisfy $b_l \in B$. The set $\mathcal{B}$ subsequently represents all feasible combinations of bit-widths across the entire network, where each element corresponds to a complete assignment $\{b_1, b_2, \dots, b_L\}$.

When formulated as a combinatorial search problem, the bit allocation space grows exponentially with the number of blocks, reaching $|\mathcal{B}| = k^L$ in the worst case. For example, with $B = \{2, 4, 8\}$ and a model with $L = 50$ quantizable blocks, the search space is $3^{50} \approx 7.12 \times 10^{23}$, rendering exhaustive exploration computationally intractable.

To reduce the search space, we adopt a Pareto frontier approach guided by the sensitivity list. Blocks are first sorted

in descending order of their sensitivity scores, and the search is reformulated as a partition problem over the sorted list, restricting assignments to monotonically non-increasing order with respect to sensitivity rank. The reduced search space is then expressed as:

$$|\mathcal{B}| = \sum_{j=1}^k \binom{k}{j} \cdot \binom{L-1}{j-1} \quad (7)$$

where j indexes the number of distinct bit-width levels used, $\binom{k}{j}$ counts the ways to select j bit-widths from k candidates, and $\binom{L-1}{j-1}$ counts the ways to partition L sorted blocks into j contiguous groups. This formulation reduces the search space from exponential to polynomial in L , making the search computationally feasible. For the same example ($L = 50$, $k = 3$), Eq. (7) yields $|\mathcal{B}| = 3 + 147 + 1176 = 1326$, a reduction of roughly 21 orders of magnitude compared with the exhaustive search space of $3^{50} \approx 7.12 \times 10^{23}$. In our implementation with $k = 3$, we enumerate the three-level partitions of the sorted list (the $j = 3$ terms), which dominate this reduced space. Each candidate allocation is evaluated by the objective in Eq. (6), and the assignment that minimizes the accumulated sensitivity while satisfying the model-size budget is selected as the final bit-width configuration.

E. Step 4: Task-Constraint Block-Wise Optimization

After acquiring the optimal bit-width allocation for each block in Step 3, to further improve the quantization performance, we adopt task-constraint block-wise optimization, which can be divided by optimization target into two parts: scaling optimization (Section III-E1) and rounding optimization (Section III-E2).

1) *Scaling Optimization*: We apply block-wise quantization to determine the optimal scale factors for weights and biases. The scale factors for the l -th block are obtained by minimizing the discrepancy between the quantized and full-precision rate-distortion loss:

$$s_w^l, s_b^l = \underset{s_w^l, s_b^l}{\operatorname{argmin}} \left(\hat{J}(\hat{w}^l, \hat{b}^l) - J_0(w^l, b^l) \right)^2 \quad (8)$$

where $\hat{J}(\hat{w}^l, \hat{b}^l)$ denotes the rate-distortion loss evaluated with the quantized weights $\hat{w}^l$ and biases $\hat{b}^l$ of the l -th block, $J_0(w^l, b^l)$ denotes the loss of the full-precision counterpart, and s_w^l, s_b^l are the scale factors for the weights and biases of the l -th block, respectively. The optimization proceeds sequentially from the first block to the last. When optimizing the l -th block, all preceding blocks have already been quantized with fixed parameters, while all subsequent blocks retain their full-precision parameters.

2) *Rounding Optimization*: Rounding-to-nearest is a common default in PTQ but is suboptimal in general. Following AdaRound [13], AdaQuant [15], and BRECQ [14], we adopt an adaptive rounding strategy in which a learnable variable V is introduced to parameterize the rounding of weights, formally expressed as:

$$\underset{V}{\operatorname{argmin}} \left\| \Lambda(w^l x^l) - \Lambda(\hat{w}^l x^l) \right\|^2 + \beta f_{\text{reg}}(V) \quad (9)$$

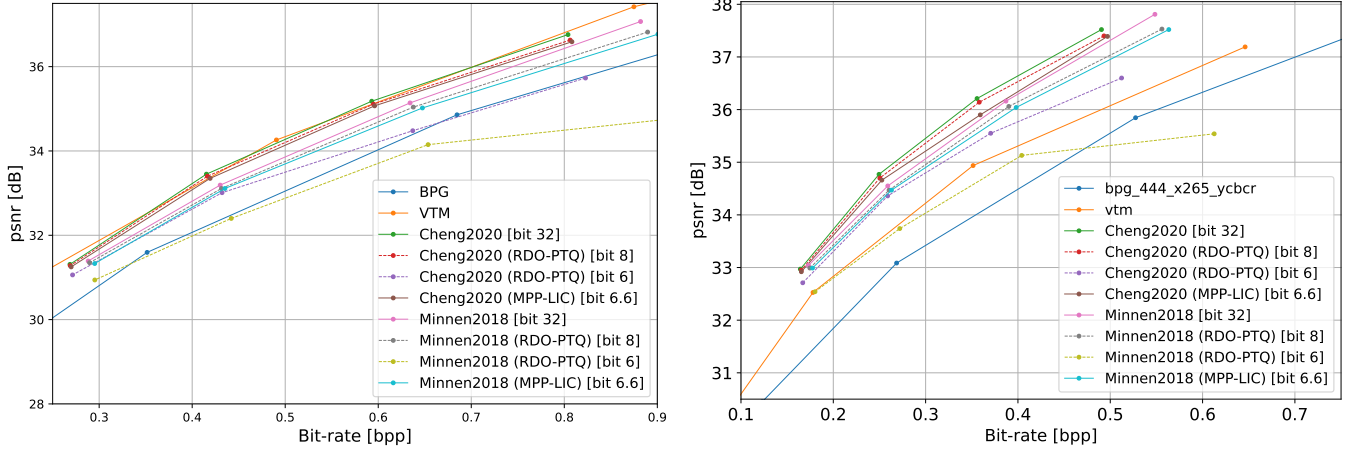


Fig. 2. Compression performance evaluation on the Kodak dataset (left) and the CLIC dataset (right). Traditional codecs and unquantized LIC models are represented by solid lines, while quantized LIC models are shown with dashed lines, with quantization bits indicated in brackets [-].

where $\Lambda(\cdot)$ denotes the activation function and $\beta f_{\text{reg}}(\mathbf{V})$ is a differentiable regularization term, weighted by β , that encourages the rounded weights $\hat{\mathbf{w}}$ to converge to integer values [13].

The per-block quantization loss is accordingly defined as:

$$\mathcal{L}_{lq} = \left\| \Lambda(\mathbf{w}^l \mathbf{x}^l) - \Lambda(\hat{\mathbf{w}}^l \hat{\mathbf{x}}^l) \right\|^2 + \beta f_{\text{reg}}(\mathbf{V}) \quad (10)$$

where $\hat{\mathbf{x}}^l$ denotes the quantized input to the l -th block, accounting for the accumulated quantization error from preceding blocks.

The total optimization loss jointly combines the task-level loss and the block-wise quantization loss:

$$\mathbf{s}^*, \mathbf{V}^* = \arg \min_{\mathbf{s}, \mathbf{V}} \lambda_t \mathcal{L}_{\text{task}} + \mathcal{L}_{lq} \quad (11)$$

where $\mathbf{s}$ denotes the collection of scale factors for both weights and biases across all blocks, and λ_t is a balancing hyperparameter empirically set to 1. The resulting mixed-precision quantized model $\mathcal{M}_{mp}$ is obtained with the optimized scale factors $\mathbf{s}^*$, rounding variables $\mathbf{V}^*$, and per-block bit-width allocation $\{b_l\}_{l=1}^L$.

IV. EXPERIMENTS

In this section, experimental settings and datasets are first described in Sections IV-A. Subsequently, we conduct experiments to justify the superiority of the proposed algorithm.

A. Implementation Details

We first introduce the pretrained LIC models in Section IV-A1. Next, we display training details of the proposed HAMP-LIC in Section IV-A2. Then, comparison methods and datasets are described in Section IV-A3.

1) *Pretrained FP32 LICs*: Two representative LIC models are selected for evaluation: *Minnen2018* [24] and *Cheng2020* [25]. Both models are based on a hierarchical variational autoencoder (VAE) architecture. Minnen2018 introduced an autoregressive context model that, for the first time, brought LIC performance to a level comparable with

the traditional image codec BPG/H.265. Cheng2020 replaced the single Gaussian prior with a discretized Gaussian mixture model, achieving performance comparable with VVC/H.266.

Full-precision pre-trained models are obtained from the CompressAI library [26]. Both models are trained using MSE as the distortion metric in the R-D loss, formulated as $\lambda \cdot 255^2 \cdot D + R$, where the Lagrange multiplier λ is selected from $\{0.0067, 0.013, 0.025, 0.0483\}$, corresponding to four different rate-distortion operating points.

2) *Training Setup*: Training is performed with a calibration dataset, which only consists of 12 images randomly selected from the CLIC dataset. The proposed framework is optimized using the ADAM [27] optimizer with a learning rate of 0.001. The mini-batch size is 4, and the input image during training is cropped to 256×256 . The optimization iterations for each block are 20,000. For the Hessian-trace estimation in Step III-B, we adopt Hutchinson's method with 5 Rademacher random vectors per block over 12 calibration samples (mini-batch size 4).

3) *Comparison Methods and Datasets*: For quantization methods, we make comparisons with both fixed-precision and mixed-precision PTQ methods. The fixed-precision baselines include Range-Adaptive Quantization (RAQ) [3], task-oriented PTQ (RDO-PTQ) [5], and the fixed-precision variant (FPQ) reported in [6]. The mixed-precision baseline is Flexible Mixed Precision Quantization (FMPQ) [6]. Among these, RDO-PTQ and FMPQ are specifically designed for LIC models: RDO-PTQ is proposed for fixed-point PTQ, while FMPQ targets mixed-precision PTQ but is limited to an 8-bit weight quantization target. The proposed HAMP-LIC overcomes this limitation and achieves superior performance.

For traditional image coders, we compare our results against the H.265/HEVC and H.266/VVC standards, using BPG software [28] and the VTM 23.11 test software [29], respectively. Both codecs were configured in all-intra mode with 8-bit YCbCr 4:4:4.

Three benchmark datasets are used for evaluation: Kodak [30], Tecnick [31], and CLIC [32]. The Kodak dataset consists of 24 uncompressed images at 768×512 resolution.

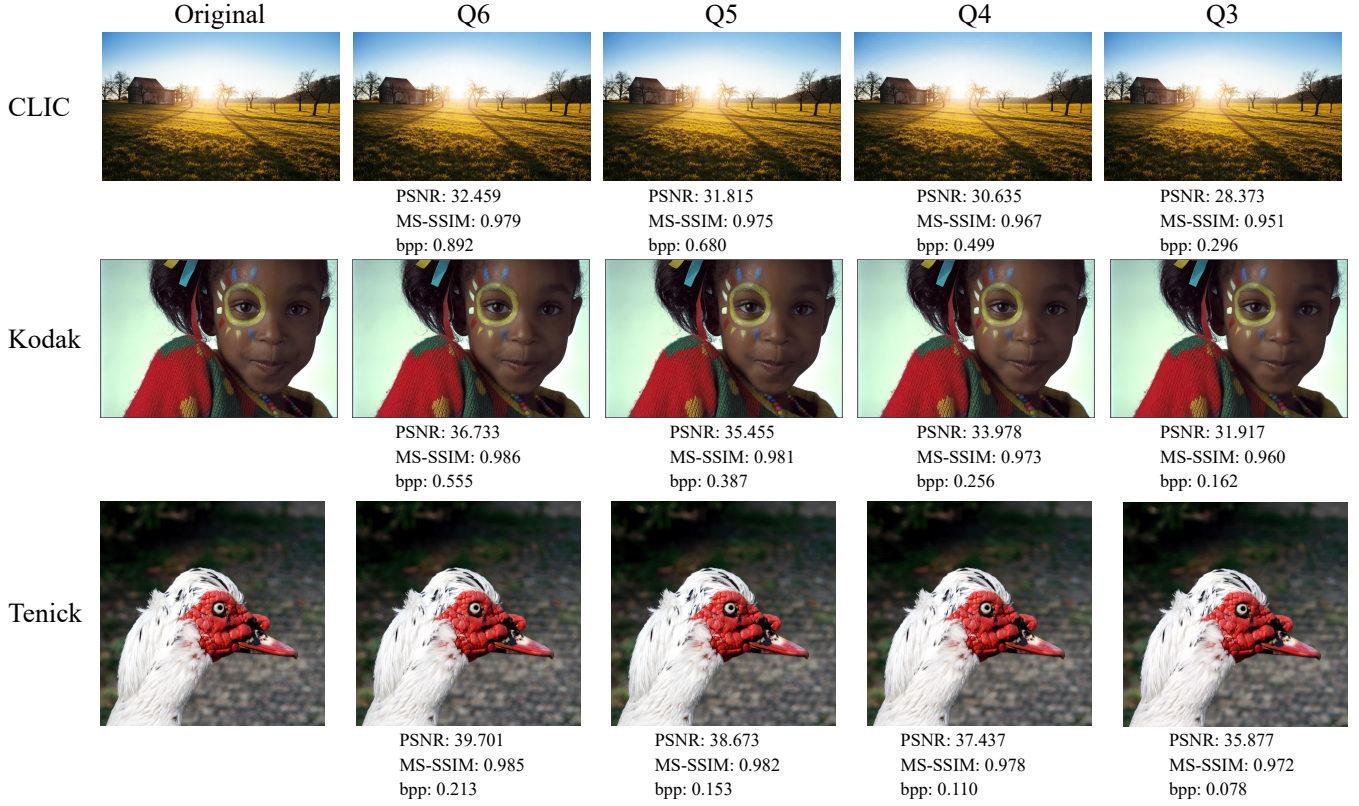


Fig. 3. Visual comparison of the proposed HAMP-LIC method across different quality levels (Q6 to Q3) on CLIC, Kodak, and Tecnick datasets. PSNR (dB), MS-SSIM, and bit-rate (bpp) are reported for each reconstruction.

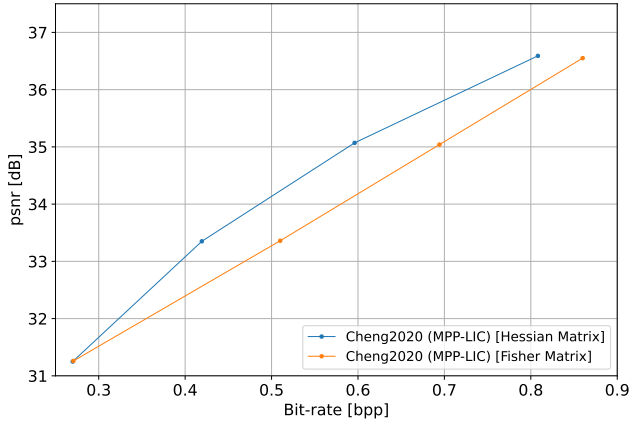


Fig. 4. Comparison of Hessian trace with Fisher matrix as the sensitivity metric used in Step 1.

The Tecnick dataset contains 100 high-resolution images at 1200×1200 . The CLIC dataset refers to the professional validation set of the CLIC 2020 challenge, comprising 41 high-quality images with diverse content and resolutions.

B. Evaluation

In this section, we present the experimental results of the proposed HAMP-LIC method, including R-D performance (Section IV-B1), BD-rate comparison (Section IV-B2),

subjective visual comparison (Section IV-B3), and encoding/decoding error rate (Section IV-B4).

1) *R-D Performance*: We evaluate the rate-distortion (R-D) efficiency by plotting PSNR (dB) against bit-rate (bpp) in Fig. 2. The comparison includes traditional codecs, full-precision anchors, and PTQ-quantized LIC models across the Kodak and CLIC datasets.

The R-D curves demonstrate that our proposed HAMP-LIC method closely aligns with the floating-point models, showing negligible performance degradation even at ultra-low bit-rates. Compared to uniform PTQ and prior RDO-based approaches, HAMP-LIC achieves a superior R-D frontier, effectively narrowing the gap between quantized and full-precision performance. Furthermore, our method remains competitive with the VVC (VTM) standard. These results confirm that the mixed-precision strategy successfully preserves the model's representation power while significantly enhancing computational efficiency for practical deployment.

2) *BD-Rate Comparison*: The proposed method is evaluated in comparison with existing PTQ approaches, using BD-rate loss relative to the full-precision models on two benchmark datasets, namely Kodak and Tecnick. As shown in Table I, the proposed HAMP-LIC achieves the highest model compression ratio (4.85 $\times$) among all compared methods. With full-precision activations ($w=6.6$, $a=32$), it also attains the lowest BD-rate loss on the *Cheng2020* model (0.59% on Kodak and 1.79% on Tecnick), outperforming the mixed-precision FMPQ baseline (0.89% and 2.68%) despite the more

Model	Method	BD-Rate Loss (%)		Compression Ratio
		Kodak	Tecnick	
Cheng2020	RAQ	27.84	29.95	4x
	RDO-PTQ	2.95	5.09	4x
	FPQ	2.05	4.97	4x
	FMPQ	0.89	2.68	3.99x
	Proposed HAMP-LIC (w=6.6, a=32)	0.59	1.79	4.85x
	Proposed HAMP-LIC (w=6.6, a=8)	2.99	6.33	4.85x
Minnen2018	RAQ	30.41	31.55	4x
	RDO-PTQ	1.19	4.01	4x
	FPQ	3.54	5.78	4x
	FMPQ	1.2	2.64	3.97x
	Proposed HAMP-LIC (w=6.6, a=32)	1.23	3.75	4.85x
	Proposed HAMP-LIC (w=6.6, a=8)	4.93	5.08	4.85x

TABLE I

BD-RATE LOSS OF VARIOUS QUANTIZATION METHODS RELATIVE TO THE ORIGINAL FULL-PRECISION (FP32) LIC MODEL. THE “X” DENOTES THE COMPRESSION RATIO OF THE WEIGHTS; FOR INSTANCE, “4X” CORRESPONDS TO QUANTIZATION FROM 32-BIT FLOATING-POINT TO 8-BIT INTEGER PRECISION.

Model	FP32 Model		Proposed HAMP-LIC	
	CPU/GPU	GPU/CPU	CPU/GPU	GPU/CPU
Encode/Decode Device	CPU/GPU	GPU/CPU	CPU/GPU	GPU/CPU
Error Rate on Kodak	24/24	21/24	0/24	0/24
Error Rate on Tecnick	98/100	97/100	0/100	0/100

TABLE II

DECODING ERROR RATES OF *Cheng2020* DURING INFERENCE WITH AND WITHOUT PTQ. THE ENCODING PROCESS IS EXECUTED ON ONE PLATFORM (CPU OR GPU), WHEREAS DECODING IS CONDUCTED ON THE OTHER (GPU OR CPU), WHERE CPU/GPU DENOTES ENCODING ON THE CPU AND DECODING ON THE GPU.

aggressive weight compression, and it remains competitive on the *Minnen2018* model.

We also evaluate the proposed HAMP-LIC method with varying activation quantization bit widths. While quantizing activations does not reduce model size, it does impact compression performance. When reducing activation precision from full FP32 to 8-bit, the BD-rate loss increases by 2.40% and 4.54% for the *Cheng2020* model on the Kodak and Tecnick datasets, respectively, and by 3.70% and 1.33% for the *Minnen2018* model. The resulting fully quantized configuration (w=6.6, a=8) trades this modest BD-rate loss for reduced computation (see the BOPS analysis in Section IV-C4) and deterministic cross-platform decoding (Section IV-B4).

These results indicate that HAMP-LIC preserves reconstruction quality at a substantially higher weight compression ratio than alternative quantization techniques such as RDO-PTQ and prior mixed-precision methods.

3) *Subjective Visual Comparison:* In Fig. 3, we present the subjective visual comparison across the CLIC, Kodak, and Tecnick datasets. The visualizations compare Original uncompressed images against reconstructions at decreasing quality levels from Q6 to Q3.

The proposed method preserves complex textures and color fidelity exceptionally well at ultra-low bit allocations. As quality decreases to Q3, bitrates drop significantly to 0.296 bpp, 0.162 bpp, and 0.078 bpp. Despite extreme compression, visual degradation is minimal, avoiding severe artifacts like

blocking or over-smoothing. This high perceptual quality is supported by strong MS-SSIM scores of 0.951, 0.960, and 0.972 at Q3, directly corroborating the superior compression efficiency demonstrated in our BD-rate analysis.

4) *Encoding/Decoding Error Rate:* As shown in Table II, the FP32 model suffers from significant decoding errors when encoding and decoding are performed on different platforms (CPU/GPU or GPU/CPU). This indicates a severe lack of robustness and platform independence, likely caused by numerical non-determinism between the different hardware architectures (e.g., subtle differences in floating-point computation on CPUs vs. GPUs) [2], [4]. In contrast, the Proposed HAMP-LIC method reduces the error rate to zero on both datasets and for both cross-platform scenarios, ensuring reliable decoding regardless of the hardware used for encoding or decoding.

C. Ablation Study

1) *Sensitivity Metric Analysis:* To further investigate the efficacy of various sensitivity measures, we conduct experiments using the Fisher information matrix as a substitute for the Hessian matrix. As presented in Fig. 4, it reveals that utilizing the Hessian matrix yields superior results compared to the Fisher matrix under the same bit-rate restriction. Second-order metrics like the Hessian are superior to first-order gradients because they capture the curvature of the loss landscape, not just its slope [7]. This provides a more accurate model of each

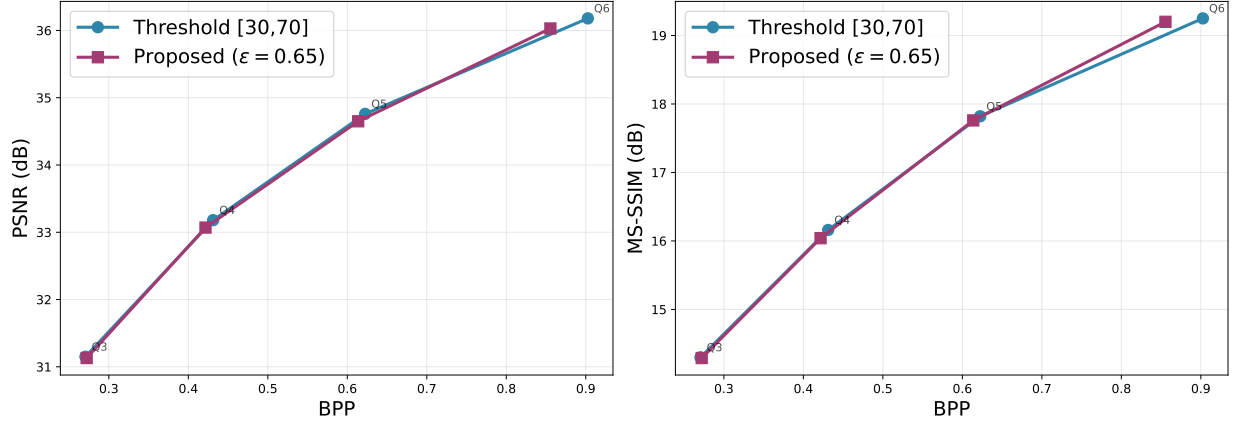


Fig. 5. Comparison of rate distortion performance between the proposed method ($\epsilon = 0.65$, average bit-width of 5.719) and the threshold-based baseline (boundaries at 30% and 70%, average bit-width of 6).

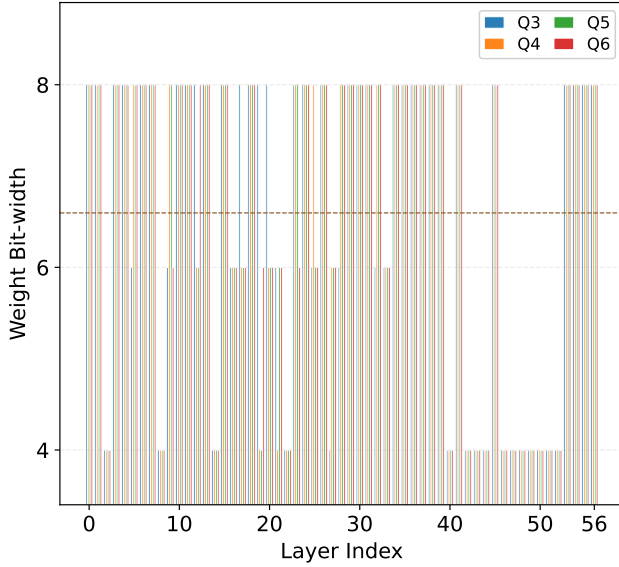


Fig. 6. Bit-width distribution across the *Cheng2020* model layers for four different quality levels.

TABLE III
BOPS COMPARISON ACROSS QUALITY LEVELS (INPUT RESOLUTION 512×768 , $b^a = 8$).

Quality	λ	MACs (G)	BOPS-Mixed (G)	BOPS-FP32 (G)	vs. W8A8
Q3	0.0067	34.3	2007.5	35138.5	↓8.6%
Q4	0.0250	76.9	4628.2	78780.0	↓6.0%
Q5	0.0250	76.9	4652.6	78780.0	↓5.5%
Q6	0.0483	76.9	4661.9	78780.0	↓5.3%

parameter’s sensitivity, leading to more effective optimization and higher-performance quantization.

2) *Bit Allocation Methods*: To evaluate the bit allocation strategy from Step III-D, we compare the proposed optimization in Equation 6 against a baseline threshold approach. In the baseline, bits are assigned by thresholding the sensitivity

list: the most sensitive 30 percent of blocks receive 8 bits, the middle 40 percent receive 6 bits, and the least sensitive 30 percent receive 4 bits, yielding an average bit-width of 6 bits. Conversely, the proposed method achieves a lower average bit-width of 5.719 with ϵ set to 0.65. Experimental results in Figure 5 show that the proposed method maintains rate distortion performance consistent with the baseline, even outperforming it at higher bitrates ($\text{bpp} > 0.6$). These findings demonstrate that the proposed optimization effectively reduces average bit weight while preserving reconstruction quality.

3) *Bit Distribution for Each Layer*: We conduct a study of the bit-width distribution across the *Cheng2020* model quantized via our HAMP-LIC method. In Fig. 6, we plot these distributions for four quality levels. The *Cheng2020* model contains 57 quantizable layers: layers [0–18] comprise the main encoder (g_a), [19–40] the main decoder (g_s), [41–45] the hyper-encoder (h_a), [46–52] the hyper-decoder (h_s), and [53–56] the entropy model (entropy parameters and context prediction). We observe that main-path weights (layers [0–40]) are generally more sensitive to quantization, requiring higher bit-widths than hyper-path weights. Specifically, the last layers of the encoder (layer 18) and decoder (layer 39) necessitate high bit-widths, as they encode critical information regarding the latent representation and the reconstructed output image, respectively.

4) *BOPS Analysis*: To further quantify the computational savings of our FMPQ method, we evaluate the Bit Operations (BOPS) for each quality level, defined as:

$$\text{BOPS} = \sum_i \text{MACs}_i \times b_i^w \times b^a, \quad (12)$$

where MACs_i and b_i^w denote the multiply-accumulate operations and weight bit-width of layer i , respectively, and $b^a = 8$ is the fixed activation bit-width. As reported in Table III, our mixed-precision quantization achieves BOPS of 2007.5 G, 4628.2 G, 4652.6 G, and 4661.9 G for quality levels $\lambda \in \{0.0067, 0.025, 0.025, 0.0483\}$, corresponding to reductions of 94.3%, 94.1%, 94.1%, and 94.1% relative to the full-precision (FP32) baseline. Compared to uniform 8-bit weight quantization (W8A8), our method further reduces

BOPS by 8.6%, 6.0%, 5.5%, and 5.3%, respectively. This gain stems directly from the bit-width allocation described above: the hyper-path layers (h_a and h_s , layers [41–52]), which are identified as less sensitive by our Hessian-based metric, are predominantly assigned 4-bit weights, whereas W8A8 applies a fixed 8-bit budget uniformly. The main-path layers (g_a and g_s , layers [0–40]) retain higher bit-widths to preserve rate-distortion performance, contributing the majority of total BOPS. These results confirm that our sensitivity-aware bit allocation not only maintains reconstruction quality but also yields a measurable reduction in computational cost beyond that achieved by simple uniform low-bit quantization.

V. CONCLUSION

In this paper, we presented HAMP-LIC, a Hessian-aware mixed-precision post-training quantization framework for deploying learned image compression models on resource-constrained hardware. HAMP-LIC combines Hessian-trace-based block sensitivity with a task-aware rate-distortion criterion, allocates bit widths through an efficient Pareto-frontier search, and applies block-wise reconstruction to reduce quantization error. Experiments on the *Minnen2018* and *Cheng2020* models across multiple benchmark datasets show that HAMP-LIC achieves up to $4.85\times$ model compression with negligible BD-rate loss, outperforms existing fixed- and mixed-precision PTQ methods, and completely eliminates cross-platform encoding-decoding errors. Future work will extend HAMP-LIC to transformer-based LIC architectures, automate the selection of the compression-ratio hyperparameter ϵ , and reduce calibration overhead.

REFERENCES

- [1] D. He, Z. Yang, Y. Chen, Q. Zhang, H. Qin, and Y. Wang, “Post-training quantization for cross-platform learned image compression,” *arXiv preprint arXiv:2202.07513*, 2022.
- [2] J. Ballé, N. Johnston, and D. Minnen, “Integer networks for data compression with latent-variable models,” in *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2019. [Online]. Available: <https://openreview.net/forum?id=S1zz2i0cY7>
- [3] W. Hong, T. Chen, M. Lu, S. Pu, and Z. Ma, “Efficient neural image decoding via fixed-point inference,” *IEEE Transactions on Circuits and Systems for Video Technology (TCSVT)*, vol. 31, no. 9, pp. 3618–3630, 2021.
- [4] D. He, Z. Yang, Y. Chen, Q. Zhang, H. Qin, and Y. Wang, “Post-training quantization for cross-platform learned image compression,” *CoRR*, vol. abs/2202.07513, 2022.
- [5] J. Shi, M. Lu, and Z. Ma, “Rate-distortion optimized post-training quantization for learned image compression,” *IEEE Transactions on Circuits and Systems for Video Technology (TCSVT)*, vol. 34, no. 5, pp. 3082–3095, 2024.
- [6] M. A. F. Hossain, Z. Duan, and F. M. Zhu, “Flexible mixed precision quantization for learned image compression,” *IEEE International Conference on Multimedia and Expo (ICME)*, pp. 1–8, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:272999263>
- [7] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, “HAWQ: hessian aware quantization of neural networks with mixed-precision,” in *International Conference on Computer Vision (ICCV)*. IEEE, 2019, pp. 293–302.
- [8] Y. Zhang, W. Shen, T. Gao, N. Zhang, and P. Wang, “Mpp-lic: Mixed precision post-training quantization for learned image compression,” in *2026 Data Compression Conference (DCC)*, 2026, pp. 487–487.
- [9] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [10] S. Jung, C. Son, S. Lee, J. Son, J. Han, Y. Kwak, and S. J. Hwang, “Learning to quantize deep networks by optimizing quantization intervals with task loss,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [11] R. Banner, Y. Nahshan, E. Hoffer, and D. Soudry, “Post training 4-bit quantization of convolutional networks for rapid-deployment,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [12] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [13] M. Nagel, R. A. Amjad, M. van Baalen, C. Louizos, and T. Blankevoort, “Up or down? adaptive rounding for post-training quantization,” in *International Conference on Machine Learning (ICML)*, 2020.
- [14] Y. Li, R. Gong, X. Tan, Y. Yang, P. Hu, Q. Zhang, F. Yu, W. Wang, and S. Gu, “BRECQ: pushing the limit of post-training quantization by block reconstruction,” in *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2021. [Online]. Available: <https://openreview.net/forum?id=POWv6hDd9XH>
- [15] I. Hubara, Y. Nahshan, Y. Hanani, R. Banner, and D. Soudry, “Improving post training neural quantization: Layer-wise calibration and integer programming,” *CoRR*, vol. abs/2006.10518, 2020.
- [16] X. Wei, R. Gong, Y. Li, X. Liu, and F. Yu, “Qdrop: Randomly dropping quantization for extremely low-bit post-training quantization,” in *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2022.
- [17] H. Sun, L. Yu, and J. Katto, “Q-LIC: quantizing learned image compression with channel splitting,” *IEEE Transactions on Circuits and Systems for Video Technology (TCSVT)*, vol. 35, no. 4, pp. 3798–3811, 2025.
- [18] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, “Haq: Hardware-aware automated quantization with mixed precision,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [19] B. Wu, Y. Wang, P. Zhang, Y. Tian, P. Vajda, and K. Keutzer, “Mixed precision quantization of convnets via differentiable neural architecture search,” *CoRR*, vol. abs/1812.00090, 2018.
- [20] Y. Zhou, S. Moosavi-Dezfooli, N. Cheung, and P. Frossard, “Adaptive quantization for deep neural network,” in *Association for the Advancement of Artificial Intelligence (AAAI)*. AAAI Press, 2018, pp. 4596–4604.
- [21] X. Huang, Z. Shen, S. Li, Z. Liu, X. Hu, J. Wicaksana, E. P. Xing, and K. Cheng, “SDQ: stochastic differentiable quantization with mixed precision,” in *International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 9295–9309.
- [22] L. Yang and Q. Jin, “Fracbits: Mixed precision quantization via fractional bit-widths,” in *Association for the Advancement of Artificial Intelligence (AAAI)*. AAAI Press, 2021, pp. 10 612–10 620.
- [23] Z. Dong, Z. Yao, Y. Cai, D. Arfeen, A. Gholami, M. W. Mahoney, and K. Keutzer, “HAWQ-V2: Hessian aware trace-weighted quantization of neural networks,” in *Neural Information Processing Systems (NIPS)*, vol. 33, 2020, pp. 18 518–18 529.
- [24] D. Minnen, J. Ballé, and G. Toderici, “Joint autoregressive and hierarchical priors for learned image compression,” *Neural Information Processing Systems (NIPS)*, 2018.
- [25] Z. Cheng, H. Sun, M. Takeuchi, and J. Katto, “Learned image compression with discretized gaussian mixture likelihoods and attention modules,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 7936–7945.
- [26] J. Bégin, F. Racap’e, S. Feltman, and A. Pushparaja, “Compressai: a pytorch library and evaluation platform for end-to-end compression research,” *ArXiv*, vol. abs/2011.03029, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:226254313>
- [27] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *International Conference for Learning Representations (ICLR)*, 2015.
- [28] Bellard, “Bpg image format,” <https://bellard.org/bpg/>, 2014.
- [29] JVET, “Vvc test model (vtm),” https://vcgit.hhi.fraunhofer.de/jvet/VVCSsoftware_VTM, 2018.
- [30] E. K. Company, “Kodak lossless true color image suite,” <http://r0k.us/graphics/kodak/>, 1999.
- [31] N. Asuni and A. Giachetti, “Testimages: A large data archive for display and algorithm testing,” *J. Graph. Tools*, vol. 17, pp. 113–125, 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:8082012>
- [32] T. George, S. Wenzhe, T. Radu, T. Lucas, B. Johannes, A. Eirikur, J. Nick, and M. Fabian, “Workshop and challenge on learned image compression (clic2020),” *CVPR*, 2020. [Online]. Available: <http://www.compression.cc>